Supplementary Material

A DERIVATION OF THE EXPONENTIAL BIAS INVERSE CDF

We derive the closed-form inverse CDF for the index-based exponential bias sampler used in Tempest, along with its numerically stable large-n approximation.

A.1 Setup

Consider a temporal neighborhood $\Gamma = \{e_0, e_1, \ldots, e_{n-1}\}$ sorted in increasing timestamp order. The sampling probability for the i-th edge is proportional to e^i:

$$P(i) \propto e^i, \quad i \in \{0, 1, \ldots, n-1\}.$$

A.2 Normalization

The unnormalized weights are:

$$w_i = e^i.$$

The total weight is the finite geometric series:

$$Z = \sum_{k=0}^{n-1} e^k = \frac{e^n - 1}{e - 1}.$$

Thus, the normalized probability is:

$$P(i) = \frac{e^i}{Z} = \frac{(e-1)\, e^i}{e^n - 1}.$$

A.3 CDF

The cumulative distribution function (CDF) up to and including index i is:

$$F(i) = \sum_{k=0}^{i} P(k) = \frac{e^{i+1} - 1}{e^n - 1},$$

using the geometric series sum $\sum_{k=0}^{i} e^k = \frac{e^{i+1}-1}{e-1}$ and cancelling $(e-1)$.

A.4 Inverse Transform Sampling

Inverse transform sampling sets:

$$u = F(i) = \frac{e^{i+1} - 1}{e^n - 1}, \quad u \sim \mathcal{U}(0, 1).$$

Multiplying by $(e^n - 1)$:

$$u(e^n - 1) = e^{i+1} - 1.$$

Adding 1 to both sides:

$$1 + u(e^n - 1) = e^{i+1}.$$

A.5 Solving for i

Taking the natural logarithm:

$$\ln(1 + u(e^n - 1)) = i + 1.$$

Rearranging:

$$i = \ln(1 + u(e^n - 1)) - 1.$$

Since i is an integer index:

$$\boxed{i = \lfloor \ln(1 + u(e^n - 1)) - 1 \rfloor} \tag{1}$$

which matches Equation 5 in the main text.

A.6 Large-n Approximation

When n is large, $e^n \gg 1$ and $e^n - 1 \approx e^n$. The logarithm argument becomes:

$$1 + u(e^n - 1) \approx u\, e^n.$$

Thus:

$$i \approx \ln(u\, e^n) - 1 = \ln(u) + n - 1.$$

This yields the numerically stable approximation:

$$\boxed{i \approx \lfloor n + \ln(u) - 1 \rfloor} \tag{2}$$

matching Equation 6 in the main text.

A.7 Numerical Stability Note

For large n, computing e^n directly may overflow in `float32` (overflow for $n \gtrsim 88$) or `float64` ($n \gtrsim 709$). The form in Equation 2 avoids overflow by separating n additively from $\ln(u)$.

A.8 Worked Example

Let $n = 100$, $u = 0.37$. The exact form (Eq. 1) gives:

$$\begin{aligned} i &= \lfloor \ln(1 + 0.37(e^{100} - 1)) - 1 \rfloor \\ &= \lfloor \ln(1 + 0.37 \times 2.69 \times 10^{43}) - 1 \rfloor \\ &= 98. \end{aligned}$$

Using the large-n form (Eq. 2):

$$i \approx \lfloor 100 + \ln(0.37) - 1 \rfloor = \lfloor 100 - 0.994 - 1 \rfloor = 98.$$

Both forms agree, but the stable form avoids computing $e^{100} = 2.69 \times 10^{43}$ directly.

B DERIVATION OF THE LINEAR BIAS INVERSE CDF

We derive the closed-form inverse CDF for the index-based linear bias sampler.

B.1 Setup

For a temporal neighborhood of length n with indices $i \in \{0, 1, \ldots, n-1\}$, assign linear weights proportional to position:

$$w_i = i + 1.$$

The total weight is:

$$Z = \sum_{k=0}^{n-1} (k + 1) = \frac{n(n+1)}{2}.$$

B.2 Random draw in raw-weight space

Draw a random value r uniformly from $[0, Z)$:

$$r \sim \mathcal{U}([0, Z)).$$

B.3 Solving for i

The index i is found by solving:

$$\frac{i(i+1)}{2} = r \quad \Longleftrightarrow \quad i^2 + i - 2r = 0.$$

Applying the quadratic formula and taking the positive root:

$$i = \frac{-1 + \sqrt{1 + 8r}}{2}.$$

Since i must be an integer index, we take the floor:

$$\boxed{i = \left\lfloor \frac{-1 + \sqrt{1 + 8r}}{2} \right\rfloor} \tag{3}$$

B.4 Normalized-u form

If instead we draw $u \sim \mathcal{U}([0,1))$ and set $r = uZ$ with $Z = \frac{n(n+1)}{2}$, substituting into (3) gives:

$$i = \left\lfloor \frac{-1 + \sqrt{1 + 8u \cdot \frac{n(n+1)}{2}}}{2} \right\rfloor = \left\lfloor \frac{-1 + \sqrt{1 + 4u\, n(n+1)}}{2} \right\rfloor.$$

This is the normalized form used when working with $u \in [0,1)$.

C PER-HOP EDGE PICKER PSEUDOCODE

Algorithm 1 Temporal Edge Picker (Per-Hop)

Require: `curr_node`, `prev_ts`, `walk_dir`, `picker_type`
Obtain contiguous edge block $[l, r)$ for `curr_node`
if `walk_dir` == FORWARD **then**
  $c \leftarrow$ first index in $[l, r)$ with $t >$ `prev_ts` {binary search}
  Candidate range $\leftarrow [c, r)$
else
  $c \leftarrow$ first index in $[l, r)$ with $t \geq$ `prev_ts`
  Candidate range $\leftarrow [l, c)$
end if
if Candidate range is empty **then**
  return NULL
end if
Locate timestamp groups within the candidate range
Perform hierarchical sampling
if `picker_type` is index-based **then**
  Sample a timestamp group using the inverse-CDF sampler
else
  Sample a timestamp group using cumulative weights and binary search
end if
Select an edge uniformly from the sampled timestamp group
return selected edge

This section presents the pseudocode for the per-hop edge selection procedure executed by each walk thread during progression. The procedure operates on the node- and timestamp-grouped view described in the subsection *Node- and Timestamp-Grouped View* of the main paper, where edges incident to each node are stored contiguously and ordered by timestamp. At each hop, a thread:

(1) Identifies the contiguous edge block corresponding to the current node.
(2) Locates the temporal cutoff via binary search to derive the causality-preserving range.
(3) Performs *hierarchical sampling*.
  - Selects a timestamp group from the candidate range according to the chosen temporal bias.
  - Samples an edge from the chosen group. For first-order walks this selection is uniform, while for second-order Temporal Node2Vec walks it is weighted by the structural bias $\beta(u, w)$.

Algorithm 1 shows the full per-hop procedure. This procedure performs no per-hop neighborhood enumeration. The temporal constraint is enforced solely through the binary-search cutoff, after which sampling operates within a contiguous memory slice.

D STREAMING INGESTION AND INDEX CONSTRUCTION

This section describes the procedure used to ingest new temporal edges and rebuild the indexing structures used during walk sampling. Edges arrive in batches and are sorted by timestamp before being merged into the global edge store. If sliding-window execution is enabled, edges older than the current window are removed. The system then rebuilds the auxiliary indices used during walk execution. First, the timestamp-grouped view is constructed by grouping edges with identical timestamps and computing cumulative weights for weighted temporal sampling. Next, the node-and-timestamp-grouped view is built by sorting edge indices by node and timestamp. Node-level offsets and node–timestamp group offsets are then computed. These structures enable efficient temporal filtering and edge selection during walk progression. The overall procedure is summarized in Algorithm 2.

Algorithm 2 Streaming Ingestion and Index Construction

Require: New edge batch $B = \{(u_i, v_i, t_i)\}$, window size Δ
Append and Maintain Temporal Window
Sort edges in B by timestamp
Merge B into the global edge store
$t_{\max} \leftarrow \max_i(t_i)$
Remove edges where $t < t_{\max} - \Delta$
Build Timestamp-Grouped View (Global Temporal Access)
Compute unique timestamp groups and offsets
Compute cumulative weights per timestamp group for weighted temporal sampling
Build Node-and-Timestamp-Grouped View (Temporal Neighborhoods)
Sort edge indices by source node and timestamp
Sort edge indices by target node and timestamp
Compute node-group offsets
Compute timestamp-group counts per node
Compute node–timestamp group offsets
Compute cumulative weights for node–timestamp groups

E IMPLEMENTATION NOTES

Edges are stored in a structure-of-arrays layout consisting of separate contiguous arrays for source nodes, target nodes, and timestamps. This layout improves memory locality and enables coalesced memory access during GPU traversal. Auxiliary arrays store active node identifiers, timestamp-group offsets, and optional cumulative weight values used by weighted temporal samplers.

Tempest supports both CPU and GPU execution. When GPU mode is enabled, the active temporal graph and its index structures reside in device memory, while edge batches are initially ingested on the host and then copied to device buffers. Memory allocation and copying are handled through unified helper routines that select host or device operations depending on the execution backend.

Walk generation kernels are specialized by graph direction (directed or undirected), walk direction (forward or backward), and temporal bias type. The runtime selects the appropriate kernel variant based on the configured walk parameters.

Generated walks are stored in dense arrays of size [num_walks× max_walk_length] for node identifiers and timestamps. A separate array records the effective length of each walk. Walks that terminate early are padded with sentinel values.

Index construction relies on parallel primitives including sorting, prefix-sum scans, and group-boundary detection. These operations materialize the timestamp-grouped and node-and-timestamp-grouped views used during walk sampling.

F TEMPORAL BIAS FUNCTION VALIDATION

To validate the behavior of different temporal bias functions and their impact on walk structure, we constructed a synthetic temporal graph with 1,000 nodes and 1 million edges where node participation follows a Gaussian distribution centered on the middle nodes (nodes around ID 500). Specifically, we generated edge endpoints by sampling from a normal distribution with mean $\mu = n_nodes/2$ and standard deviation $\sigma = n_{\text{nodes}}/6$, creating higher connectivity for central nodes and lower connectivity for peripheral nodes. We then sampled 1 million walks of maximum length 100 using three edge pickers: `Exponential`, `Linear`, and `Uniform`. Each plot compares the visitation frequency of nodes in the sampled walks with the original edge distribution at the node level.

Results. Figure 1 shows that the `Exponential` picker produces walks that most closely match the original distribution. This bias favors edges with timestamps close to the current walk time, which keeps walks concentrated in temporally dense regions—leading to fewer terminations and longer average lengths (96.3 steps). As a result, the visitation frequencies closely align with the original bell-shaped structure. In contrast, `Linear` picker explores more broadly but still with some bias toward newer edges. This results in shorter walks (avg. length: 15.3) and slightly flatter distributions. The `Uniform` picker samples edges randomly without regard to time, which causes frequent walk termination and a more scattered node visitation profile (avg. length: 10.0). All three strategies retain the overall shape of the input distribution, validating that Tempest's walk sampling mechanisms preserve structural fidelity while respecting temporal causality. The choice of bias model provides fine-grained control over locality vs. exploration.

G DOWNSTREAM TASK: FRAUD DETECTION

In this section, we evaluated Tempest's downstream utility for fraud detection on the DGraph dataset (Table 1 in the main text), a temporal financial interaction graph in which each node has 17-dimensional features and approximately 1.3% of nodes are labeled as fraudsters.

G.0.1 Methodology. We built temporal node embeddings with Tempest random walks and skip-gram Word2Vec: 20 walks per node (10 forward, 10 backward), max walk length 50, and ExponentialIndex edge picker. In streaming mode, edges were processed in batches of 100 timestamps with a sliding window of 400 timestamps. We fused embeddings with tabular features and a small set of node-level graph metrics (log-degree, recency, burstiness, neighbor diversity) via a lightweight MLP. Training used 50 epochs, class-weighted binary cross-entropy, AdamW, cosine learning-rate schedule, and early stopping on validation AUC.

Results. Test ROC-AUC performance (mean ± standard deviation across $n = 5$ independent runs) is reported in Table 1.

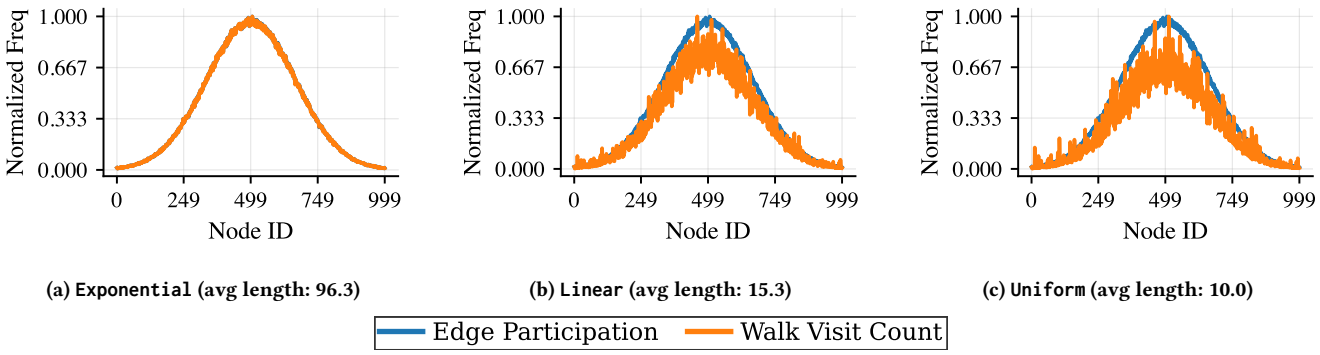
Method	**ROC–AUC (test)**
Ours (fusion MLP)	
Tempest (Streaming) + Node + Graph feats	0.7826 ± 0.0003
Tempest (Full) + Node + Graph feats	0.7844 ± 0.0007
Node + Graph feats	0.7211 ± 0.0006
Tempest (Streaming) + Node feats	0.7252 ± 0.0015
Reported in DGraph [2]	
TGAT (dynamic GNN) [3]	0.792 ± 0.001
SAGE [1]	0.778 ± 0.001
GCN	0.751 ± 0.002
MLPs	0.723 ± 0.002

Table 1: Fraud detection on DGraph

G.0.2 Discussion. Unlike GraphSAGE or TGAT, which are powerful but complex end-to-end GNN pipelines trained offline on full history without incremental updates, our approach centered on Tempest's streaming walk engine that supports sliding-window ingestion and on-the-fly embedding updates. While these GNN models do not rely on walk-based sampling and therefore are not directly comparable in methodology, we include them as widely used baselines for downstream performance. Streaming attained near full-mode accuracy, while the full variant's slightly better performance required keeping the entire graph in memory. Ablations showed that the dominant lift came from Tempest's temporal embeddings, with simple node-level graph metrics being secondary.

H ADDITIONAL PERFORMANCE GAP ANALYSIS

Figure 2 shows the downstream performance of the bulk and streaming approaches under varying levels of structured noise for four metrics beyond AUC: accuracy, precision, recall, and F1-score. The trends match those in the main-text AUC plot: all metrics degrade smoothly with increasing noise, and the streaming curves track the bulk curves closely with small average gaps.



(a) `Exponential` (avg length: 96.3)

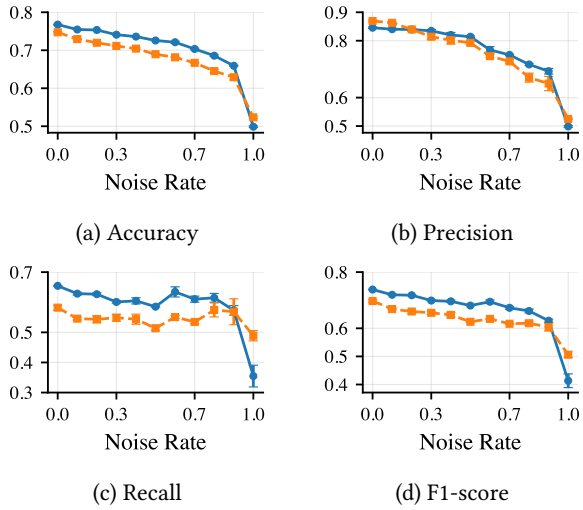
(b) `Linear` (avg length: 15.3)

(c) `Uniform` (avg length: 10.0)

Figure 1: Node visitation frequency compared to the original edge distribution on a synthetic graph.

Table 2: Mean and standard deviation of the performance gap (bulk – streaming) across noise levels.

Metric	Mean Gap	Std Dev
AUC	0.0408	0.0163
Accuracy	0.0271	0.0173
Precision	0.0111	0.0246
Recall	0.0449	0.0609
F1-score	0.0355	0.0419



(a) Accuracy

(b) Precision

(c) Recall

(d) F1-score

Figure 2: Downstream performance under structured noise for bulk and streaming approaches across four metrics: (a) Accuracy, (b) Precision, (c) Recall, (d) F1-score. The trends match the AUC plot in the main text.

Table 2 summarizes the mean and standard deviation of the gaps (bulk – streaming) for all five metrics across all tested noise levels.

REFERENCES

[1] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30.

[2] Xuanwen Huang, Yang Yang, Yang Wang, Chunping Wang, Zhisheng Zhang, Jiarong Xu, Lei Chen, and Michalis Vazirgiannis. 2022. DGraph: a large-scale financial dataset for graph anomaly detection. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) *(NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 1654, 13 pages.

[3] Da Xu, Chuanwei Ruan, Evren Körpeoglu, Sushant Kumar, and Kannan Achan. 2020. Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks. In *Proceedings of the 8th International Conference on Learning Representations*. TGAT - Temporal Graph Attention Network.